

esp32_sensor

ESP32 sensor firmware for the [mir](#) project. Exposes a small text-based command interface over Serial (USB/UART) and TCP, plus a binary sensor-data stream on a dedicated TCP port and a UDP discovery responder.

Build

- Framework: Arduino via [PlatformIO](#)
- Build: `pio run`
- Upload: `pio run -t upload`
- Serial baud: 115200

Transports

Transport	Port	Use
Serial	—	Configuration + diagnostics
TCP	5000	Command interface (text, newline-terminated)
TCP data	5001	Sensor frame stream (binary, opaque)
UDP disc.	5679	Discovery request → reply on 5678

Frame format

Commands are terminated by a single `\n` (LF). Arguments are `key=value` pairs separated by `;`:

```
<command> <key1>=<value1>;<key2>=<value2>\n
```

Responses are serialised on the same stream as:

```
#<command> <key>=<value>;<key>=<value>\n
```

A successful response always begins with `status=success`. Errors use `status=error;error=<reason>`.

Commands

Sensor data (Serial or TCP)

Command	Args	Response
<code>get_data</code>	none	<code>status=success;<sensor fields></code> (depends on type)

Command	Args	Response
<code>get_data_schema</code>	none	<code>status=success;<schema descriptor></code>

General configuration (Serial or TCP)

Command	Result
<code>get_configuration</code>	see fields below (no WiFi credentials)
<code>set_configuration</code>	updates the listed fields and persists to NVS namespace <code>sensor_cfg</code>

Fields returned by `get_configuration` and accepted by `set_configuration`:

Key	Type	Notes
<code>sensor_type</code>	string	<code>simulation</code> , <code>lidar</code> , <code>wheather</code>
<code>sensor_id</code>	string (≤ 31 chars)	free-form identifier
<code>ap1_ip</code>	IPv4 or <code>auto</code>	static IP when joining AP #1 (<code>auto</code> = DHCP)
<code>ap2_ip</code>	IPv4 or <code>auto</code>	static IP when joining AP #2 (<code>auto</code> = DHCP)
<code>wifi_timeout</code>	int (≥ 2)	seconds before giving up an AP connection
<code>loop_period</code>	int ($10 \leq x \leq 999$)	main loop period in ms
<code>current_ip</code>	IPv4	current DHCP/static IP (read-only)

The credentials `ap1_ssid`, `ap1_pwd`, `ap2_ssid`, `ap2_pwd` are **not** exposed by `get_configuration` and **not** accepted by `set_configuration`. Any occurrence in a TCP request is silently ignored.

AP credentials — **Serial-only**

Command	Notes
<code>get_ap_configuration</code>	returns <code>ap1_ssid</code> , <code>ap1_pwd</code> , <code>ap2_ssid</code> , <code>ap2_pwd</code>
<code>set_ap_configuration</code>	updates the same four fields and persists to NVS

These are the **only** commands that touch SSIDs and WiFi passwords.

Source restriction. Any TCP request for `set_ap_configuration` or `get_ap_configuration` is rejected before dispatch:

```
<cmd> set_ap_configuration ap1_ssid=foo;ap1_pwd=bar\n
#set_ap_configuration status=error;error=serial_only_command
```

The same call from Serial succeeds:

```
<cmd> set_ap_configuration ap1_ssid=foo;ap1_pwd=bar\n#set_ap_configuration status=success
```

This is deliberate: credentials must not transit over the WiFi link itself, where they would be readable from any device on the same network.

System (Serial or TCP)

Command	Effect
---------	--------

<code>reboot</code>	restarts the ESP32 (response sent first)
---------------------	--

Examples

TCP / Serial — set static IP for primary AP:

```
set_configuration ap1_ip=192.168.1.42;ap2_ip=auto\n#set_configuration status=success
```

Serial only — set WiFi credentials:

```
set_ap_configuration ap1_ssid=MyNetwork;ap1_pwd=secret\n#set_ap_configuration status=success
```

TCP — same call is refused:

```
set_ap_configuration ap1_ssid=MyNetwork;ap1_pwd=secret\n#set_ap_configuration status=error;error=serial_only_command
```

Read the full (non-credential) configuration:

```
get_configuration\n#get_configuration\nsensor_type=lidar;sensor_id=esp_0;ap1_ip=192.168.1.42;ap2_ip=auto;wifi_time\nout=5;loop_period=25;current_ip=192.168.1.42
```

Read just the WiFi credentials (Serial only):

```
get_ap_configuration\n#get_ap_configuration ap1_ssid=MyNetwork;ap1_pwd=secret;ap2_ssid=;ap2_pwd=
```

Persistence and boot

- All configuration lives in NVS namespace `sensor_cfg`.
- At boot the device:
 1. Loads configuration from NVS.
 2. Prints the configuration to Serial (credentials included — console is trusted).
 3. Tries to join `ap1`, then `ap2` if the first fails.
 4. Starts the TCP command port (5000), the TCP data port (5001) and the UDP discovery listener (5679).
 5. Initialises the sensor, then enters the main loop.

Source map

File	Role
<code>src/main.cpp</code>	setup, loop, command dispatcher and handlers
<code>src/command.cpp</code>	text command parser (<code>key=value;...</code>)
<code>include/command.h</code>	parser API
<code>src/sensor_*.cpp</code>	per-sensor implementations
<code>src/rplidar.cpp</code>	RPLIDAR driver glue
<code>platformio.ini</code>	PlatformIO project (deps, board, build flags)